הפקולטה להנדסה ע"ש פרניק
كلية الهندسة على اسم فرنيك
PERNICK FACULTY OF ENGINEERING

הנדסה. עיצוב. אמנות.
هندسة. تصميم. فن.
ENGINEERING. DESIGN. ART.

שנקר
شنكار
SHENKAR

NavSight

Software Engineering B.Sc. Final Project

Software Design Document

Authors:

Roey Ben Harush (315676163)

Tamir Sobuh (206029084)

Morad Zubidat (208156828)

Supervisor:

Mr. Amit Dunsky

15/01/2026

Approved by: _____ Date: _____

Approved by: _____ Date: _____

Content

1. Introduction

a. System Overview

NavSight is a self-contained, real-time mobile navigation system designed for Android devices. It leverages Visual-Inertial Odometry (VIO) combined with Dead Reckoning (DR) to provide continuous position tracking without dependence on Global Navigation Satellite Systems (GNSS). The system is engineered to operate entirely offline, processing sensor data locally on the device to deliver a responsive and private navigation experience. Its primary use case is urban micromobility, enabling users to navigate environments where GPS signals are unreliable or unavailable, such as dense urban canyons, tunnels, and indoor spaces.

b. Purpose

The purpose of the NavSight system is to overcome the limitations of traditional satellite-based navigation in challenging environments. GNSS signals are often blocked, reflected, or degraded in urban canyons created by tall buildings, inside tunnels and underground passages, and within indoor environments. NavSight provides a reliable navigation alternative by using the device's own camera and motion sensors to track movement relative to a starting point, ensuring users can navigate confidently regardless of satellite signal availability.

c. Scope

In-Scope: The project encompasses the development of a Visual Odometry (VO) pipeline for processing camera frames, a Dead Reckoning (DR) module utilizing Inertial Measurement Unit (IMU) data, a sensor fusion algorithm to combine VO and DR estimates, a native Android application with a real-time path visualization UI, and complete offline operation with no network dependency.

Out-of-Scope: The project does not include full 3D Simultaneous Localization and Mapping (SLAM) with loop closure, cloud-based processing or data storage, integration with external map services, turn-by-turn navigation instructions, or voice guidance features.

d. Definitions and Acronyms

VIO (Visual-Inertial Odometry): A technique that estimates device motion by fusing visual information from a camera with inertial measurements from an IMU.

VO (Visual Odometry): The process of determining a device's position and orientation by analyzing sequential camera images.

DR (Dead Reckoning): A method of calculating current position by using a previously known position and advancing it based on estimated speed and direction from inertial sensors.

IMU (Inertial Measurement Unit): A sensor package typically containing accelerometers and gyroscopes that measures linear acceleration and angular velocity.

GNSS (Global Navigation Satellite System): A general term for satellite-based navigation systems, including GPS, GLONASS, Galileo, and BeiDou.

JNI (Java Native Interface): A programming framework that allows Java/Kotlin code running in the Android runtime to call and be called by native applications written in C/C++.

e. Constraints

Hardware Dependency: The system's accuracy and performance are fundamentally limited by the quality of the device's camera and IMU sensors. Lower-quality sensors will result in higher drift and less reliable tracking.

Environmental Limitations: Visual odometry requires sufficient lighting and visual texture in the environment. Performance will degrade in low-light conditions, featureless environments (e.g., blank walls), or with excessive motion blur.

Real-time Processing: All computations must complete within a strict time budget (target <200ms per frame) to provide responsive feedback. This constrains the complexity of algorithms that can be employed.

Power Consumption: Continuous use of the camera and high-frequency IMU sampling are battery-intensive. The system must be optimized to balance accuracy with acceptable battery drain for practical usage durations.

Platform: The application targets Android 10 (API level 29) and above, utilizing the Android NDK for native C++ components.

2. System Architecture

a. Architectural Description and Design: Roles, Activities and Data

The NavSight system employs a multi-layered architecture designed for modularity, performance, and clear separation of concerns.

Layer 1 - Presentation Layer (Android UI): Built with Kotlin and Jetpack Compose, this layer is responsible for rendering the camera preview, displaying the tracked path, showing system status (e.g., tracking quality, mode), and handling user interactions (start, stop, pause, resume).

Layer 2 - Application Logic Layer (Orchestrator): This Kotlin-based layer manages the application's lifecycle and state machine. It coordinates sensor data acquisition from the camera and IMU, packages data for the native layer, and receives processed pose updates to forward to the UI.

Layer 3 - Native Interface Layer (JNI Bridge): This layer provides the interface between the Kotlin application code and the C++ native processing core. It handles data marshalling, manages the lifecycle of native objects, and ensures thread-safe communication.

Layer 4 - Native Processing Core (C++): This is where the computationally intensive work occurs. It contains the Vision Processing Module for feature detection, tracking, and visual odometry. It includes the Inertial Pre-integration Module for processing IMU data. The Sensor Fusion Module combines visual and inertial estimates. The Concurrency Control Module manages parallel processing pipelines.

b. The Life Cycle of the System

The system operates through the following states:

1. Initialization: The application launches, requests necessary permissions (camera, sensors), and loads native libraries.

2. Calibration: The system performs an initial sensor warm-up and may prompt the user to perform a brief calibration motion if required.

3. Active Tracking (VIO Mode): The primary operational state. The system fuses camera and IMU data to produce continuous pose estimates. The UI displays the live path.

4. Degraded Tracking (DR Mode): If visual tracking is lost (e.g., camera covered, low light), the system automatically falls back to dead reckoning using only IMU data. A visual indicator alerts the user to the reduced accuracy.

5. Recovery: When visual features become available again, the system transitions back to full VIO mode.

6. Paused: The user can pause the session, suspending all sensor processing while retaining the current path data.

7. Termination: The user stops the session. A summary of the tracked path may be displayed, and resources are released.

3. Literature Survey

a. Problem Survey

Visual-Inertial Odometry (VIO) addresses the fundamental challenge of estimating a device's position and orientation using only onboard sensors, without relying on external infrastructure like GPS satellites. This problem is critical for navigation in GNSS-denied environments such as urban canyons, indoor spaces, tunnels, and underground facilities.

The core technical challenge lies in fusing two complementary but imperfect sensor modalities. Cameras provide rich geometric information through feature tracking but suffer from scale ambiguity, motion blur, and failure in low-texture or low-light conditions. Inertial Measurement Units (IMUs) offer high-frequency motion estimates immune to visual conditions but accumulate drift rapidly when double-integrated.

Key approaches in the literature include feature-based SLAM systems like ORB-SLAM that extract and match distinctive visual features across frames, direct methods such as LSD-SLAM that operate on image intensities, Visual Odometry approaches focusing on incremental motion estimation, and IMU pre-integration techniques for efficient sensor fusion.

b. Solution Survey

The NavSight system addresses the VIO problem by implementing a tightly-coupled sensor fusion approach that combines visual feature tracking with inertial pre-integration.

Visual Odometry Pipeline: Based on principles from established VO systems, our implementation uses sparse feature detection and KLT optical flow for inter-frame tracking, providing relative pose estimates from camera motion.

Inertial Pre-integration: Following established methodology, we pre-integrate accelerometer and gyroscope measurements between visual keyframes, efficiently handling high-frequency IMU data (100-200Hz).

Sensor Fusion Strategy: We employ a filtering approach that fuses visual pose estimates with pre-integrated IMU measurements, prioritizing real-time performance on mobile hardware.

Dead Reckoning Fallback: When visual tracking fails, the system automatically transitions to dead reckoning mode using only inertial data, providing continuity until visual tracking recovers.

c. Discussion and Conclusion

The literature review reveals a clear trade-off between accuracy and computational efficiency in VIO systems. Full SLAM solutions offer superior accuracy through loop closure and global bundle adjustment, but their computational demands make them challenging for mobile devices.

Our approach draws inspiration from lightweight VIO systems that prioritize real-time performance. By avoiding expensive operations like loop closure, we maintain the latency required for responsive navigation feedback (<200ms target). The trade-off is that positional drift will accumulate over time.

The dead reckoning fallback mechanism is crucial for practical deployment in urban environments that frequently present visual challenges. Future work could explore integration with occasional GNSS fixes to bound accumulated drift.

4. Design

a. Data Design

i. Database Description

The system is designed for complete offline operation and does not utilize a traditional persistent database. All session data is stored ephemerally in device memory during active tracking sessions. Processed metadata such as path summaries may be stored temporarily in local application files for evaluation purposes.

ii. Global Data Structures Design

The system relies on several key data structures:

Sensor Frame Packet: A composite object containing a video frame, timestamp, and IMU samples since the previous frame.

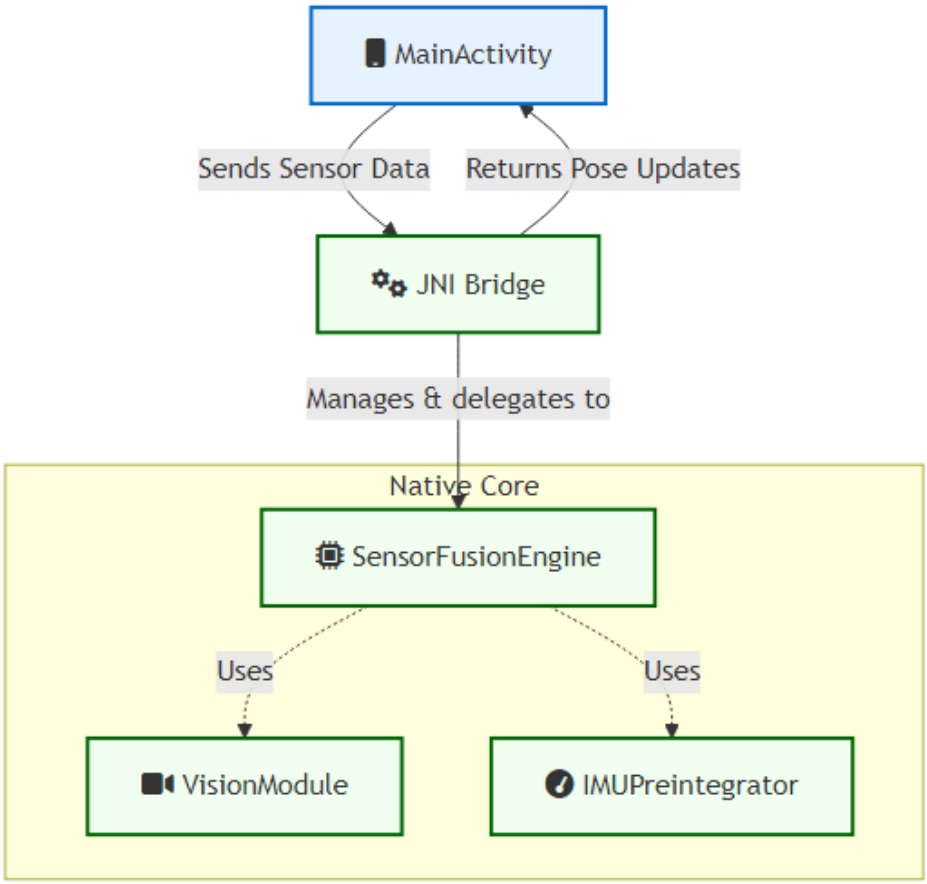
Motion State Vector: Represents estimated device state including 3D position, orientation (quaternion), velocity, and timestamp.

Tracked Feature Set: Collection of 2D points representing visual features being tracked across frames.

Path Trajectory: Ordered list of Motion State Vectors representing the user's complete path.

b. Structural Design

i. Class Diagram



c. Interactions Design

i. Use Cases

UC-1: Track a Navigation Path - The user initiates the application, calibrates the device, starts tracking, navigates their route with real-time path display, and stops to view a summary.

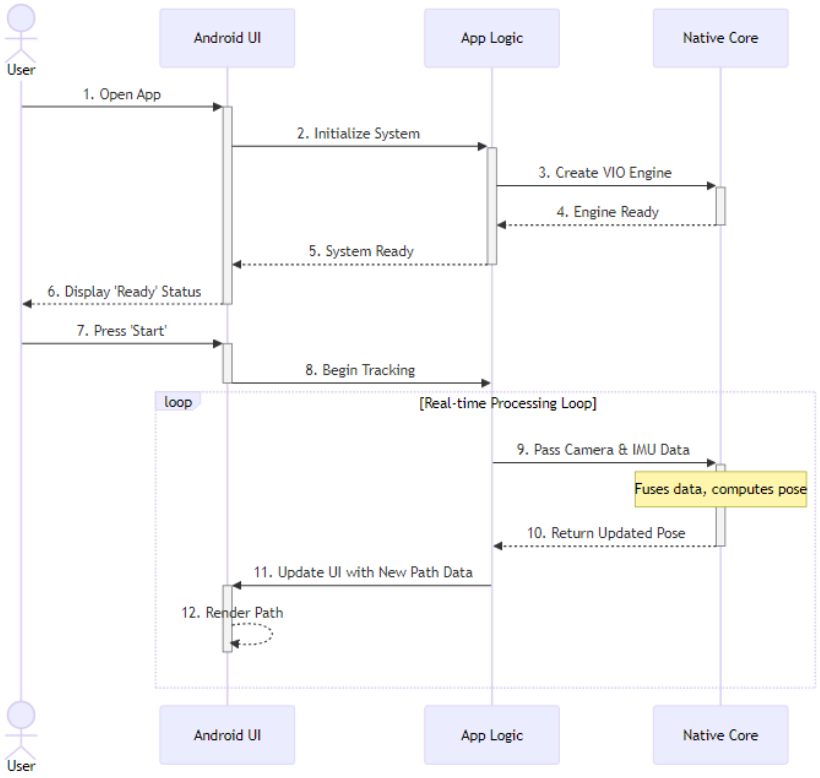
UC-2: Handle Degraded Visuals - During tracking, the user enters a poorly lit area. The system detects feature loss, switches to DR mode, notifies the user, and automatically resumes VIO when conditions improve.

UC-3: Pause and Resume - The user pauses the session mid-route, the system suspends processing, and later resumes tracking from the last known position.

ii. Sequence Diagram

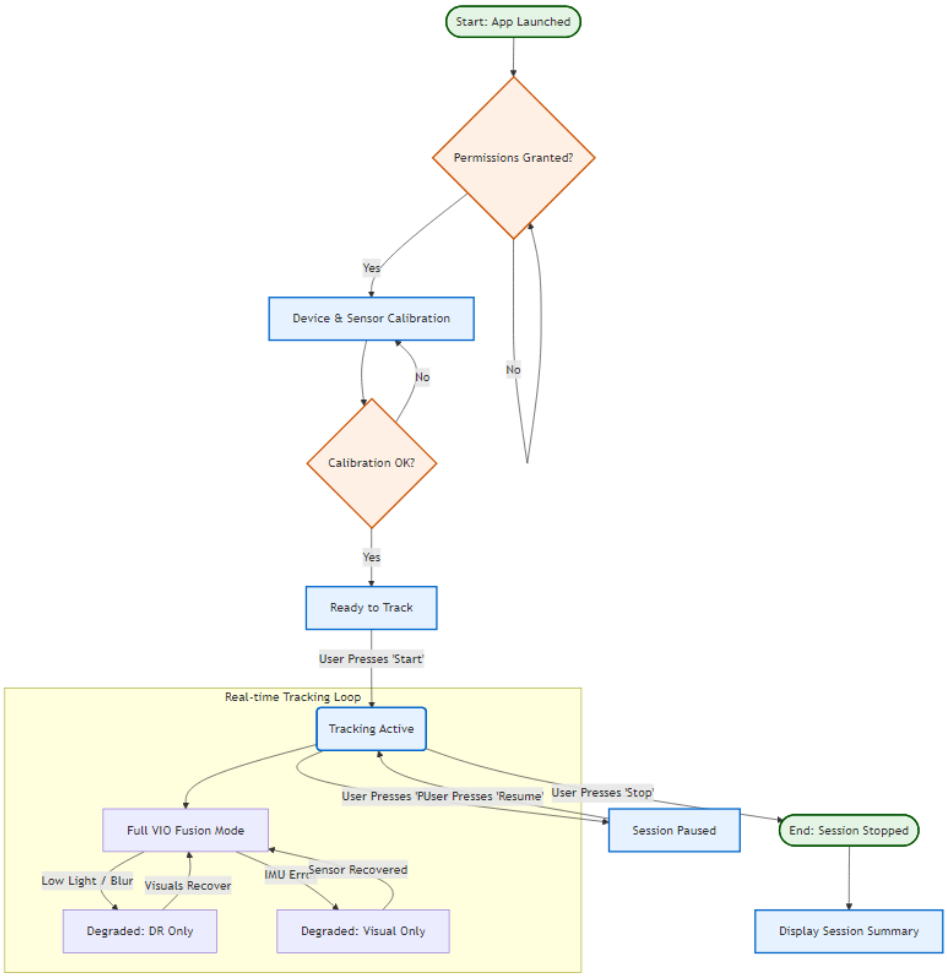
This diagram shows the sequence of interactions for processing a single sensor frame packet, from user input through the Android UI, Application Logic, and

Native Core layers.



iii. Activity Diagram

This diagram models the system's operational states and transitions, reflecting the lifecycle from app launch through permissions, calibration, active tracking, degraded modes, and session termination.



d. Software Architecture Pattern

i. Three-tier: Data, Logic and Presentation tiers

The NavSight system employs a layered architecture with four primary tiers: Presentation Layer (Android UI), Application Logic Layer, Native Interface Layer (JNI Bridge), and Native Processing Core (C++). This design ensures modularity, facilitates concurrent processing, and isolates performance-critical native code from the application framework.

ii. MVC: Model, View, Controller structure

Within the Presentation Layer, the application employs MVVM (Model-View-ViewModel) patterns common in modern Android development. The Model consists of data classes like VioData. The View comprises Jetpack Compose UI elements. The ViewModel role is served by MainActivity and associated state management logic.

e. Testing Platform

The testing strategy encompasses multiple levels:

Unit Testing: Native C++ modules tested using GoogleTest framework to validate core algorithm correctness.

Integration Testing: JNI boundary tested for correct data marshalling, thread safety, and native object lifecycle management.

Application-level Testing: Android application tested using Espresso for UI interaction and JUnit for application logic.

5. Appendix-A

a. POC discussion (if relevant)

The Proof of Concept validated core NavSight functionality.

POC Objective: Demonstrate accurate movement tracking when the device is held parallel to the ground (90 degrees), displaying trajectory on screen in real-time.

Test Scenario: Device held horizontally while walking forward, backward, left, and right. The system interprets camera feed and IMU data to calculate relative movement.

Success Criteria: Correct direction detection, real-time path updates, tracking continuity during normal walking.

Results: The POC successfully demonstrated VIO tracking with horizontal phone orientation. On-screen visualization correctly reflected movement direction, validating the implementation feasibility.

6. Appendix-B

a. Team Roles

Roey Ben Harush (Team Lead / Main Developer):
- Lead Android application development (Kotlin, Jetpack Compose)
- System architecture design and layer separation
- JNI bridge implementation
- Task coordination and integration
- UI/UX design

Tamir Sobuh (Native Core Developer):
- C++ native processing core development
- Vision Processing Module implementation
- Native code optimization
- Unit testing for native components

Morad Zubidat (Algorithm & Integration Developer):
- Inertial Pre-integration Module
- Sensor Fusion algorithm
- Sensor synchronization and calibration
- Integration testing and documentation

7. Appendix-C

a. Schedule / Gantt (possible print screen or sharable link)

Project Timeline (October 2025 - January 2026):

Phase 1 - Research & Design (October 2025):
Weeks 1-2: Literature review, technology selection
Weeks 3-4: System architecture design, SDD preparation

Phase 2 - Core Development (November 2025):
Weeks 1-2: Native C++ core, Vision Module
Weeks 3-4: IMU Pre-integration, initial sensor fusion

Phase 3 - Application Development (December 2025):
Weeks 1-2: Android UI, JNI bridge
Weeks 3-4: Layer integration, POC validation

Phase 4 - Testing & Refinement (January 2026):
Week 1: System testing, bug fixes, optimization
Week 2: Final documentation, presentation

Milestones: POC (Dec 15), Alpha (Dec 28), Final (Jan 7)

8. Appendix-D

a. System Screens (Updated version after FRS)